Hostellers

Hostel Management System

Software Requirements Specification
and UML Diagrams

PROJECT TEAM MEMBERS

- **M.Sanjay pal** - 424150
- **Raj Gireesh** - 424169
- **P.Jyothiswar** - 424165

Software Engineering Project | Academic Year 2026

National Institute of Technology Andhra Pradesh

SYSTEM CONTEXT & BOUNDARIES

1. Executive Summary, System Scope & Personas

1.1 PURPOSE & CORE PROBLEM ANALYSIS

Background: Traditional residential hostel administration at collegiate institutions relies on manual paper ledgers, physical movement gate-passes, unrecorded room maintenance, and ad-hoc communication. These procedures suffer from high vulnerability to loss, lack of accountability, and zero real-time administrative oversight.

Solution: Hostellers is an enterprise-grade web application engineered to digitize the complete hostel governance lifecycle at NIT Andhra Pradesh. The platform unifies student self-registration, transparent room exploration, democratic complaint upvoting, daily cleaning records, non-local outing requests, and formal room evacuations.

1.2 SCOPE & BOUNDARIES OF THE SYSTEM

In-Scope Capabilities:

- Role-isolated authentication for Students, Caretakers, and Administrators.
- Student registration with institutional email validation (*@student.nitandhra.ac.in).
- Democratic complaint system with peer upvoting, status tracking, and deletion.
- Daily maintenance logging (cleaning and mopping verification) for resident rooms.
- Outing permission approvals with mandatory 10-digit parent emergency phone validation.
- Room allocation & evacuation workflows governed by administrator-controlled date windows.

Out-of-Scope (Boundaries):

- Direct third-party payment gateway integration (fee receipts verified via PDF documents).
- Hardware turnstiles or physical biometric scanning apparatus at hostel gates.
- Native desktop / mobile OS binaries (standard web responsiveness covers all viewports).

1.3 USER CLASSES & ROLE-BASED RESPONSIBILITIES

STUDENT ROLE	CARETAKER ROLE	ADMINISTRATOR ROLE
Self-register using verified institutional email crede	Access assigned hostel blocks strictly scoped via	Global infrastructure authority: create, modify, and delete hostel blo
Explore hostel blocks, inspect floor layouts, and re	Review daily room cleaning records and conduct p	Provision caretaker staff accounts and assign/reassign caretakers t
Lodge maintenance complaints and upvote fellow	Review, approve, or reject non-local outing reques	Configure operational date windows for room allocation and room e
Log daily room cleaning (sweeping/mopping) and	Resolve complaints, update status flags, and dele	Review student room allocation applications and grant final approva
Apply for room allocation with fee receipts and req	Publish and delete block announcements; STRICT	Maintain emergency contact directory and inspect campus-wide oc

1.4 RUNTIME ARCHITECTURE & DESIGN CONSTRAINTS

Decoupled SPA Client:	Vanilla HTML5/CSS3/ES6+ SPA without heavy framework overhead, ensuring instant page loading.
Node.js/Express Backend:	Express REST API serving JSON endpoints, managing sessions, and enforcing role middleware.
PostgreSQL Database:	Normalized 18-table relational schema with foreign key cascades and connection pooling via pg.
Graceful Degradation:	Automated fallback to memoryStore if the database daemon drops, ensuring operational resilience.
Auxiliary Django API:	django_api/ exists as an exploratory standalone prototype, isolated from the production Express API.

SPECIFICATION DETAILS

2. Functional Requirements: Auth & Student Subsystems

2.1 REQ-AUTH: SECURE AUTHENTICATION & SESSION ISOLATION

REQ-AUTH-001	**Role-Separated Login:**	Dedicated login flows for Student (roll/email), Caretaker (emp_id/email), and Admin (key).
REQ-AUTH-002	**Bcrypt Hashing:**	Passwords hashed using bcrypt (10 rounds); database columns upgraded to TEXT to fit hashes.
REQ-AUTH-003	**HMAC-SHA256 Token:**	Cryptographically signed token format: base64url(payload) + '.' + hex(hmac_sha256).
REQ-AUTH-004	**Role Middleware:**	requireAuth(roles) strictly verifies token signature and rejects unauthorized roles with HTTP 403.
REQ-AUTH-005	**Session Lifetime:**	Frontend enforces 30-min inactivity timeout; backend accepts valid HMAC tokens up to 30 days.
REQ-AUTH-006	**Clean Placeholders:**	Input fields feature clean placeholders without confusing hardcoded default sample values.

2.2 REQ-STU: STUDENT RESIDENT SUBSYSTEM & SELF-SERVICE

REQ-STU-001

Self-Registration

- Students self-register with Roll No, Reg No, Name, Academic Year (1-4), Phone, Gender, and Institutional Email.
- Server validates institutional domain format (*@student.nitandhra.ac.in) via rigorous regex.
- System enforces database-level uniqueness on roll_number, registered_number, and email.

REQ-STU-002

Hostel & Room Exploration

- Students explore hostel blocks filtered by gender eligibility (Male, Female, Co-ed) and academic year.
- Dynamic floor maps display real-time room availability, total seater count, and occupied beds.

REQ-STU-003

Democratic Complaint Filing & Upvoting

- Students submit complaints linked to their assigned room and hostel block in hostel_complaints.
- Block peers can upvote complaints; complaint_votes table prevents duplicate votes and tracks urgency.
- Complaint status reflects real-time boolean flag (Pending / Resolved) with resolution timestamp.

REQ-STU-004

Daily Room Maintenance Logs

- Students record daily cleaning entries documenting sweeping status, mopping status, and remarks.
- Logs are stored in maintenance and hostel_maintenance tables for caretaker verification.

REQ-STU-005

Non-Local Outing Permission Requests

- Students apply for out-of-station leave specifying departure date, return date, purpose, and parent contact.
- Mandatory 10-digit parent emergency phone validation (/^[6-9][0-9]{9}$/) prevents fictitious contacts.
- Requests are routed to the student's assigned caretaker with Pending, Approved, or Rejected state.

REQ-STU-006

Room Allocation Application Workflow

- Active during administrator-scheduled allocation windows; validates academic year and gender conformity.
- Requires students to upload an official fee receipt document (PDF) for administrative verification.

REQ-STU-007

Room Evacuation Clearance Request

- Students initiate vacating requests during active evacuation windows, specifying reasons and departure date.
- Enforces multi-department clearances: room inspection, mess dues, library dues, and key handover.

GOVERNANCE & INFRASTRUCTURE

2. Functional Requirements: Caretaker, Admin & System

2.3 REQ-CAR: CARETAKER GOVERNANCE & RESIDENTIAL OVERSIGHT

REQ-CAR-001	**Assigned Block Scoping:**	Caretakers view residents, rooms, and logs only for hostels mapped to them in caretaker_hostel.
REQ-CAR-002	**Block Mutation Restriction:**	Caretakers CANNOT create, edit, or delete hostel blocks; endpoints return HTTP 403 Forbidden.
REQ-CAR-003	**Complaint Resolution:**	Caretakers inspect complaints for their blocks, toggle resolved status, and delete invalid complaints.
REQ-CAR-004	**Maintenance Inspection:**	Caretakers inspect student room cleaning logs and verify sweeping/mopping completion by floor.
REQ-CAR-005	**Outing Verification:**	Caretakers review outing requests, inspect parent contact info, and issue Approval or Rejection.
REQ-CAR-006	**Block Notice Publishing:**	Caretakers publish official announcements to their assigned block and delete outdated notices.
REQ-CAR-007	**Resident Directory:**	Caretakers view complete rosters of residents in their blocks with contact details and room assignm

2.4 REQ-ADM: INSTITUTIONAL ADMINISTRATION & INFRASTRUCTURE

REQ-ADM-001

Hostel Block Infrastructure CRUD

- Create new blocks: Block ID, Name, Floors, Seater type (e.g. 2, 4), Capacity, Gender (Male, Female, Co-ed).
- Configure floor room counts in hostel_floor_rooms and academic year eligibility in hostel_year.
- Edit block parameters and safely delete vacant blocks (blocks with active residents cannot be deleted).

REQ-ADM-002

Caretaker Staff Provisioning

- Provision caretaker staff accounts: Employee ID, Full Name, Contact Number, Email, and Bcrypt Password.
- Map caretakers to one or multiple hostel blocks in caretaker_hostel, or reassign blocks dynamically.

REQ-ADM-003

Operational Window Scheduling

- Configure start date, end date, and active status for Room Allocation and Room Evacuation windows.
- System strictly rejects student submissions occurring outside scheduled application periods.

REQ-ADM-004

Allocation & Evacuation Decisioning

- Review student allocation applications, inspect fee receipt documents, and commit students to student_hostel.
- Approve evacuation clearances upon multi-department clearance confirmation, releasing room occupancy.

REQ-ADM-005

Notice Board & Contact Directory Governance

- Publish campus-wide announcements broadcast to all hostels, or hostel-specific announcements.
- Manage campus emergency contact directory (Chief Warden, Hostel Office, Ambulance, Security).

2.5 REQ-NOT & REQ-CNT: NOTIFICATION ENGINE & EMERGENCY DIRECTORY

REQ-NOT-001	**5-Second Polling Engine:**	Client executes automated 5-second polling (setInterval 5000ms) to sync updates without page relo
REQ-NOT-002	**Status Change Alerts:**	Real-time alerts notify students of complaint resolution, outing decisions, and allocation status.
REQ-CNT-001	**Emergency Directory:**	Public & authenticated access to verified contacts: Chief Warden, Hostel Office, Security, Ambulanc
REQ-CNT-002	**Dynamic Directory Updates:**	Admin updates phone numbers, personnel names, and availability via PUT /api/admin/contacts.

POSTGRESQL SCHEMA SPECIFICATION

3. Physical Relational Database Architecture

3.1 POSTGRESQL DATA ARCHITECTURE OVERVIEW

The Hostellers persistence layer is implemented in PostgreSQL 14+ using a fully normalized relational schema comprising 18 specialized tables. Referential integrity is strictly enforced through foreign key constraints, unique keys, and ON DELETE CASCADE rules on votes and mapping records. Password fields use TEXT datatypes. Primary connection pooling is provided by Node.js pg.Pool; an in-memory memoryStore provides graceful resilience.

3.2 SPECIFICATION OF 18 RELATIONAL TABLES (database.sql)

1. admin [System administrators]
`key (PK, VARCHAR 20), password (TEXT bcrypt hash)`

2. caretaker [Hostel supervisory staff]
`employee_id (PK, VARCHAR 10), name, phone (BIGINT), email (UQ),`

3. student [Resident students]
`roll_number (PK, INT), reg_no (UQ, INT), name, year, phone, ema`

4. hostel [Hostel block buildings]
`block_id (PK, VARCHAR 10), name, floors, seater, capacity, for_`

5. hostel_year [Year-to-block eligibility]
`student_year (PK, INT), block_id (PK, FK -> hostel.block_id)`

6. hostel_floor_rooms [Rooms per floor layout]
`block_id (PK, FK -> hostel), floor_number (PK, INT), number_of_`

7. room [Individual resident rooms]
`room_id (PK, VARCHAR 20), room_number (INT), floor_number (INT)`

8. caretaker_hostel [Caretaker assigned blocks]
`employee_id (PK, FK -> caretaker), block_id (PK, FK -> hostel)`

9. student_hostel [Active room occupancy]
`roll_number (PK, FK -> student), room_id (PK, FK -> room)`

10. announcements [Notice board broadcast notices]
`announcement_id (PK, VARCHAR 20), date, time, announcement (TEXT)`

11. hostel_announcements [Hostel notice mapping]
`block_id (PK, FK -> hostel), announcement_id (PK, FK -> announcements)`

12. complaints [Maintenance complaints]
`complaint_id (PK, VARCHAR 20), roll_number (FK), room_id (FK), date, time, c`

13. hostel_complaints [Complaint block mapping]
`block_id (PK, FK -> hostel), complaint_id (PK, FK -> complaints)`

14. complaint_votes [Democratic upvotes per user]
`complaint_id (PK, FK -> complaints ON DELETE CASCADE), user_id (PK, VARCHAR`

15. maintenance [Daily room cleaning logs]
`maintenance_id (PK, VARCHAR 20), roll_number (FK), room_id (FK), cleaning_st`

16. hostel_maintenance [Cleaning block mapping]
`block_id (PK, FK -> hostel), maintenance_id (PK, FK -> maintenance)`

17. outing [Non-local outing requests]
`outing_id (PK, VARCHAR 20), roll_number (FK), from_date, to_date, purpose, p`

18. hostel_outing [Outing block mapping]
`block_id (PK, FK -> hostel), outing_id (PK, FK -> outing)`

3.3 DATA INTEGRITY, CASCADE RULES & FALLBACK STATE

- **Referential Integrity:** Foreign keys prevent orphaned complaints, maintenance logs, outings, and room allocations.
- **Cascade Delete on Votes:** complaint_votes uses ON DELETE CASCADE; deleting a complaint automatically removes all votes.
- **Multi-Table Atomic Updates:** Room allocation and room vacating operations perform atomic insertions and deletions in student_hostel.
- **In-Memory State Mirror:** memoryStore maintains live mirrors for windows (allocation, evacuation), contacts, and allocations.
- **Seed Data Alignment:** database.sql includes preloaded seed records for Vamsadhara, Nagavalli, Krishna, Sabari, and Indhravathi.

ENGINEERING STANDARDS

4. Non-Functional Quality Attributes & Caveats

4.1 SECURITY ENGINEERING & CRYPTOGRAPHY

SRS-NFR-SEC-001	**Bcrypt Credential Hashing:**	Passwords hashed with bcrypt (work factor 10); plaintext passwords never stored or logged.
SRS-NFR-SEC-002	**HMAC-SHA256 Token Signature:**	Session token signed using HMAC-SHA256 with strong server secret; tampered tokens reject
SRS-NFR-SEC-003	**Parameterized SQL Queries:**	All database operations use parameterized statements ($1, $2, ...) via pg client, eliminating SC
SRS-NFR-SEC-004	**Role Authorization Check:**	requireAuth(roles) verifies role claims; students cannot access staff endpoints; caretakers can
SRS-NFR-SEC-005	**Cross-Site Scripting Defense:**	Dynamic inputs in complaints, announcements, and maintenance remarks undergo HTML esc

4.2 PERFORMANCE, RELIABILITY & SCALABILITY

SRS-NFR-PER-001	**Sub-300ms API Latency:**	Operational JSON endpoints respond within 300ms under standard campus load of 50 concur
SRS-NFR-PER-002	**Lightweight Asset Bundle:**	Vanilla SPA bundle weight remains under 350 KB, loading in <1.2 seconds over campus Wi-F
SRS-NFR-REL-001	**Connection Pool & Failover:**	Database access managed by pg.Pool; automated fallback to memoryStore prevents unhandl
SRS-NFR-USE-001	**Responsive Viewport Layout:**	Fluid CSS Flexbox/Grid UI adapts from 360px (mobile phones) to 1920px (desktop monitors).
SRS-NFR-USE-002	**Input Validation Feedback:**	Form validation failures (phone format, email domain, fee receipt) trigger instant visual alerts.

4.3 TECHNICAL IMPLEMENTATION CAVEATS & ARCHITECTURAL RECONCILIATIONS

Custom HMAC Token vs Standard JWT:

- The backend implements custom HMAC-SHA256 signing: base64url(payload) + '.' + hex(signature).
- While cryptographically robust against client tampering, this is a two-segment custom token,
- not a 3-segment RFC 7519 JSON Web Token (JWT). It requires no external JWT dependency.

Session Expiry Divergence:

- Client enforces a 30-minute inactivity timeout via localStorage (purging token on idle timeout).
- The backend token validator accepts valid HMAC signatures up to 30 days. Direct API calls
- remain authenticated until token timestamp expires or server secret is rotated.

Database-Unavailable Fallback Architecture:

- When PostgreSQL is unreachable, backend routes degrade gracefully to an in-memory memoryStore.
- This permits offline evaluation and demonstration, but in-memory mutations reset on server restart.

Django REST Service Role (django_api/):

- The repository includes a secondary implementation under django_api/ with SQLite/PostgreSQL models.
- Code inspection confirms django_api/ is an auxiliary exploratory prototype. The production runtime
- is the Node.js/Express service on port 3000.

Transport Layer Security (TLS):

- Local development operates over HTTP without TLS. Production deployments must utilize a reverse proxy
- (NGINX / Let's Encrypt) to terminate TLS/HTTPS at the network boundary.

VERIFICATION & QUALITY ASSURANCE

5. Requirements Traceability Matrix & Test Suite

5.1 AUTOMATED REGRESSION SUITE (38 SELENIUM WEBDRIVER TEST CASES)

System requirements are fully validated by 38 automated end-to-end regression tests located in: `testing/src/test/java/com/hostel/HostelManagementTest.java. Built with Java 17+, Maven Surefire, JUnit 5,` Headless Chrome WebDriver, and direct PostgreSQL JDBC query assertions, the test suite verifies all three user roles.

Test suite coverage spans: Student (TC-01..17), Caretaker (TC-18..29), and Administrator (TC-30..38).

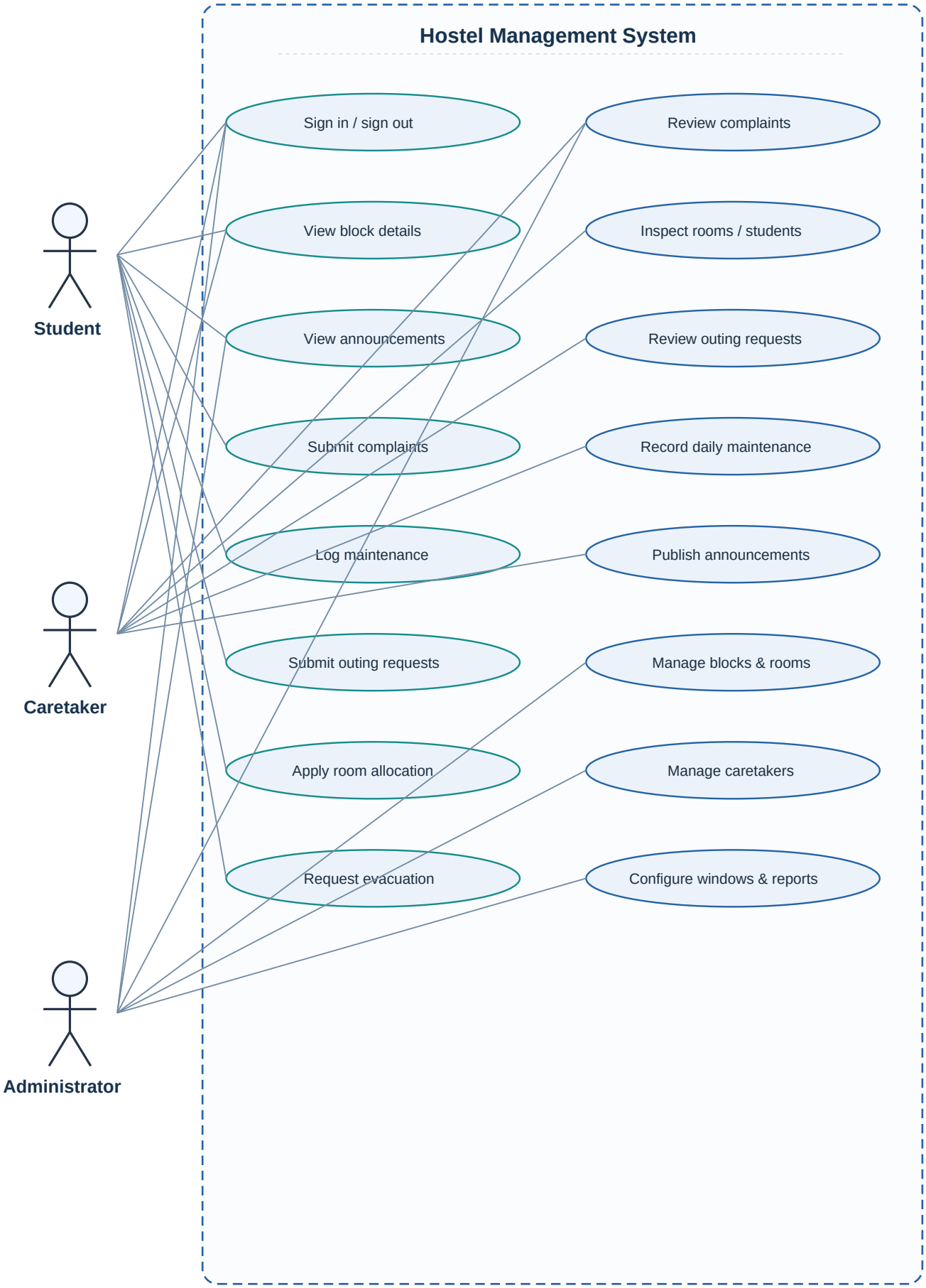
Key tests assert: student self-registration, democratic voting, caretaker block restriction (403), and window boundaries.

5.2 REQUIREMENTS TRACEABILITY MATRIX (RTM)

REQ ID	REQUIREMENT TITLE	SOURCE CODE ARTIFACT	DATABASE TABLE	TEST CASE
REQ-AUTH-001	Role-separated sign-in	`server.js:767-880`	admin, caretaker, student	TC-04, 18, 30
REQ-AUTH-002	Bcrypt password hash	`server.js:705,790`	student.password (TEXT)	TC-01, TC-04
REQ-AUTH-003	HMAC token issuance	`server.js:640-660`	Session payload signature	TC-04, 18, 30
REQ-AUTH-005	RBAC role middleware	`server.js:665-678`	Role claims validation	TC-21 (HTTP 403)
REQ-AUTH-006	30-min idle timeout	`script.js:35-50`	localStorage.lastActivity	TC-04
REQ-STU-001	Student self-registration	`server.js:689-760`	student (roll, reg, email UQ)	TC-01, TC-03
REQ-STU-004	File maintenance complaint	`server.js:1469-1550`	complaints, hostel_complaints	TC-10
REQ-STU-005	Democratic complaint vote	`server.js:1606-1680`	complaint_votes (CASCADE)	TC-11
REQ-STU-006	Daily cleaning log	`server.js:1717-1820`	maintenance, hostel_maint	TC-12
REQ-STU-007	Non-local outing request	`server.js:1824-1920`	outing, hostel_outing	TC-13, TC-14
REQ-STU-008	Room allocation application	`server.js:2525-2600`	memoryStore.allocations	TC-08, TC-09
REQ-STU-009	Room evacuation request	`server.js:2011-2026`	memoryStore.evacuations	TC-15
REQ-CAR-001	Assigned block scoping	`server.js:910-975`	caretaker_hostel	TC-20
REQ-CAR-002	Block modification forbid	`server.js:981-1110`	RBAC requireAuth(['admin'])	TC-21
REQ-CAR-003	Complaint status & delete	`server.js:1298,1552`	complaints.status	TC-22, TC-23
REQ-CAR-005	Outing review & approval	`server.js:1333-1460`	outing.status	TC-25, TC-26
REQ-CAR-006	Block notice manage	`server.js:1130,1164`	announcements, hostel_ann	TC-27, TC-28
REQ-ADM-001	Global block CRUD	`server.js:981-1110`	hostel, hostel_year, rooms	TC-31, TC-32
REQ-ADM-002	Provision caretakers	`server.js:2085-2270`	caretaker, caretaker_hostel	TC-33
REQ-ADM-003	Configure window dates	`server.js:2476-2510`	memoryStore.windows	TC-34, TC-35
REQ-ADM-004	Approve room allocation	`server.js:2322-2415`	student_hostel (commit)	TC-36
REQ-ADM-005	Approve room evacuation	`server.js:2424-2475`	student_hostel (delete)	TC-37
REQ-ADM-007	Emergency directory	`server.js:2512-2524`	memoryStore.contactUs	TC-38

Use Case View

Primary actions grouped by user role within the system boundary.



Domain Class Diagram

Core entities, mapping tables, and relationship multiplicities.

Admin	Student	Caretaker	HostelBlock
+ key + password	+ roll_number + name + email + year	+ employee_id + name + email + phone_number	+ block_id + name + floors + capacity

Room	StudentHostel	CaretakerHostel	HostelAnnouncement
+ room_id + room_number + floor_number	+ roll_number + room_id	+ employee_id + block_id	+ block_id + announcement_id

Complaint	ComplaintVote	HostelComplaint	Maintenance
+ complaint_id + status + votes + date	+ complaint_id + user_id	+ block_id + complaint_id	+ maintenance_id + cleaning_status + remarks

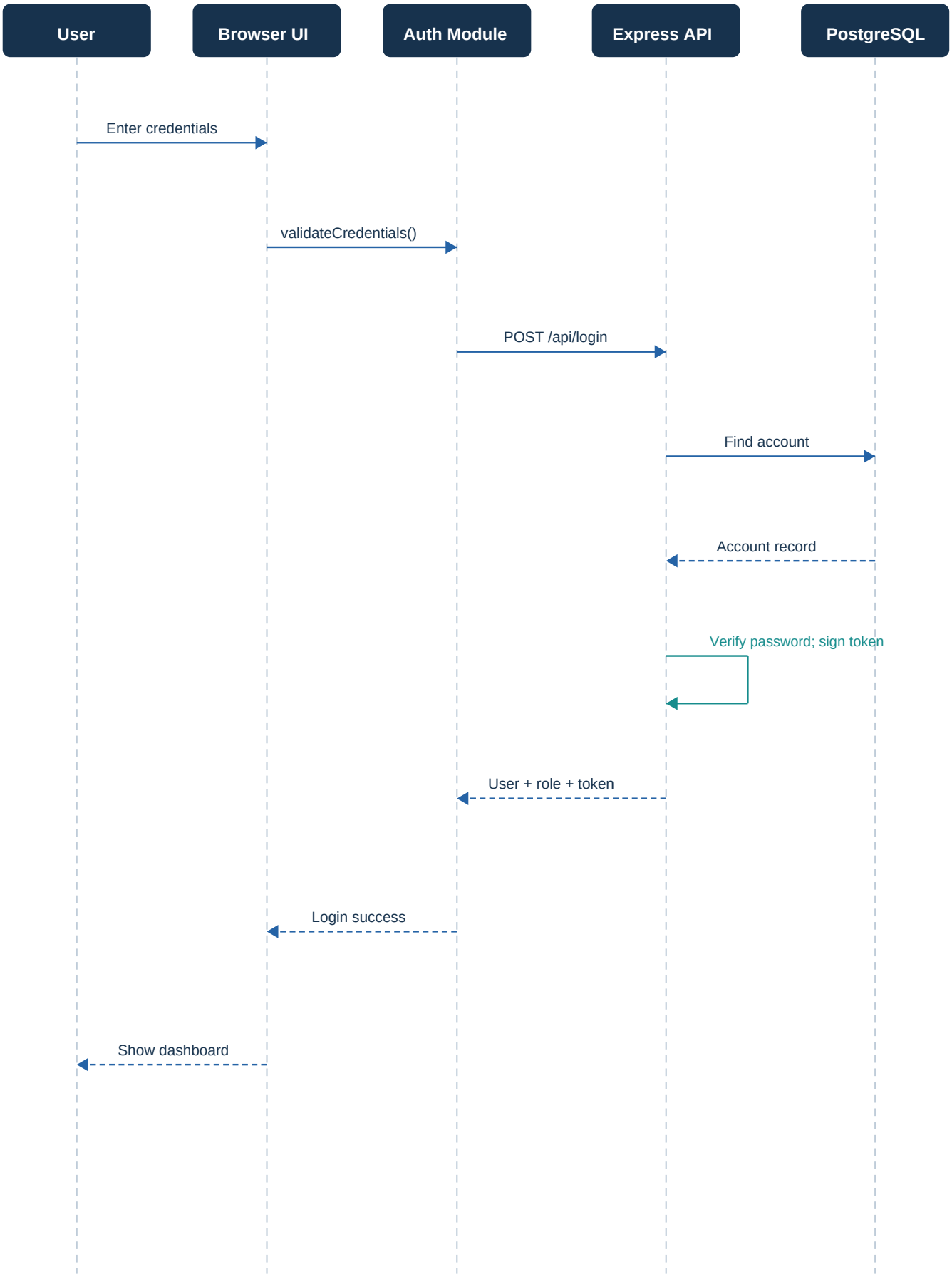
HostelMaintenance	OutingRequest	HostelOuting	Announcement
+ block_id + maintenance_id	+ outing_id + from_date + to_date + status	+ block_id + outing_id	+ announcement_id + date + text

Associations (multiplicity):

- Student 1 -- 0..* StudentHostel * -- 1 Room
- Caretaker 1 -- 0..* CaretakerHostel * -- 1 HostelBlock
- HostelBlock 1 -- 0..* Room
- Student 1 -- 0..* Complaint ; Complaint 1 -- 0..* ComplaintVote
- Student 1 -- 0..* Maintenance ; linked by HostelMaintenance
- HostelBlock 1 -- 0..* HostelYear / FloorRooms configuration
- Student 1 -- 0..* OutingRequest ; linked by HostelOuting
- HostelBlock 1 -- 0..* HostelAnnouncement * -- 1 Announcement
- HostelBlock 1 -- 0..* HostelComplaint * -- 1 Complaint
- HostelBlock 1 -- 0..* HostelMaintenance * -- 1 Maintenance
- HostelBlock 1 -- 0..* HostelOuting * -- 1 OutingRequest
- Complaint 1 -- 0..* ComplaintVote

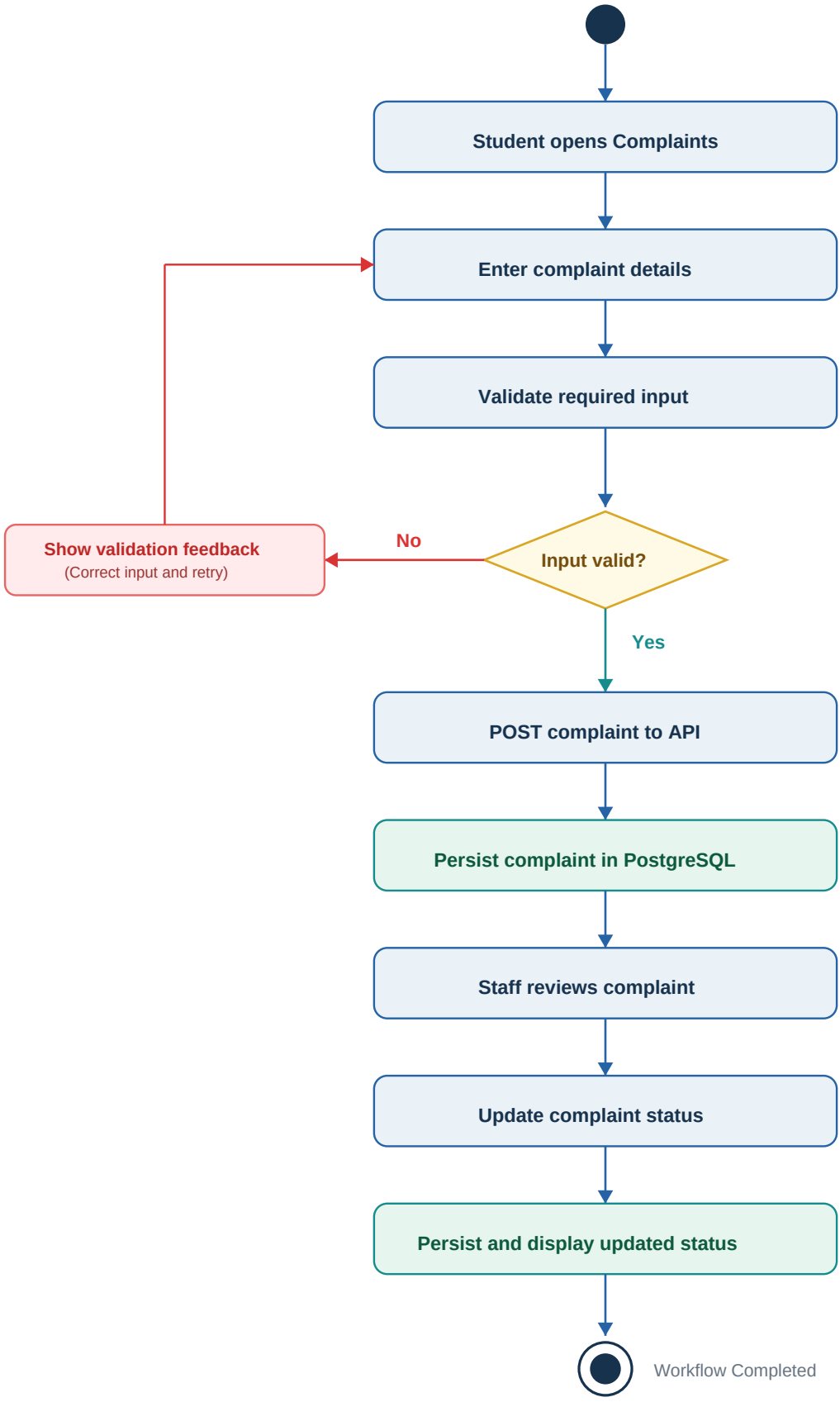
Sign-In Sequence

Successful credential validation, verification, and session creation.



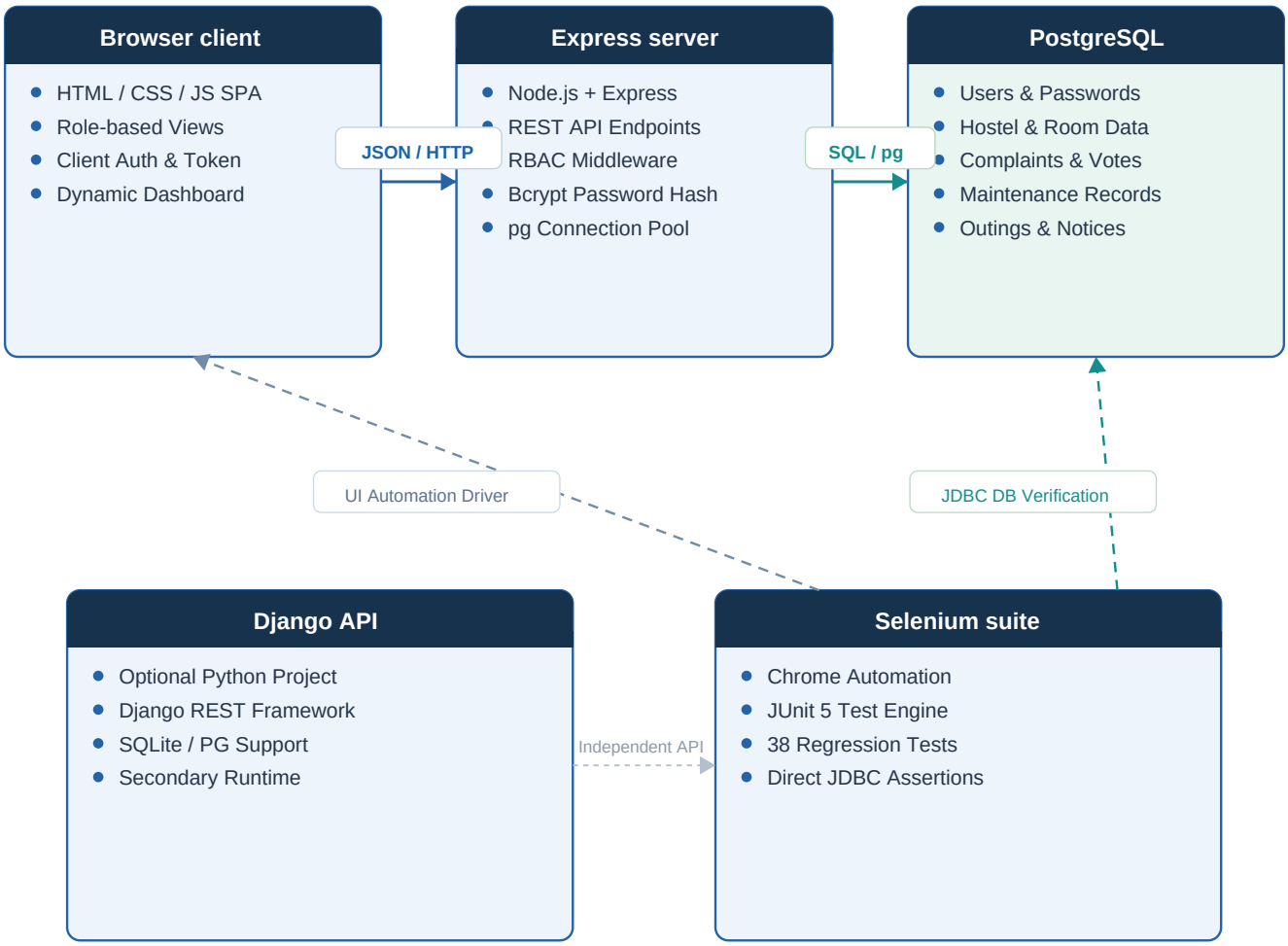
Complaint Activity

Submission validation through caretaker/admin status update workflow.



Deployment / Component View

Primary runtime components, service tiers, and supporting test suite.



System Architecture & Deployment Notes:

- The primary runtime is a lightweight Single Page Application (SPA) served by Node.js and Express.
- PostgreSQL 16 provides persistent relational storage for user accounts, room bookings, complaints, and outings.
- Password verification utilizes Bcrypt hashing; authentication tokens enforce server-side Role-Based Access Control.
- The Selenium + JUnit 5 automated regression suite verifies 38 end-to-end user flows across Student, Caretaker, and Admin roles.
- Database integrity is verified both through REST API responses and direct JDBC query assertions during test execution.